

Cookies, Sessions & URL Rewriting — Assignment Answers

Topics: Cookies, HttpSession, JSP, URL rewriting, logout, Remember Me

Q1. AddToCartServlet — Remembering Cart Items with Cookies (10 min)

Cookies are stored in the user's browser, so they survive even if the user navigates away and comes back within the cookie's lifetime (set here to 10 minutes = 600 seconds).

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class AddToCartServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String itemId = request.getParameter("itemId");

        // Check if a cart cookie already exists
        String existingItems = "";
        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("cartItems")) {
                    existingItems = c.getValue();
                }
            }
        }

        // Append the new item (URL-encode in a real app; simplified here)
        String updatedItems = existingItems.isEmpty() ? itemId : existingItems + "," + itemId;

        Cookie cartCookie = new Cookie("cartItems", updatedItems);
        cartCookie.setMaxAge(10 * 60); // 10 minutes, in seconds
        cartCookie.setPath("/"); // available across the whole app
        response.addCookie(cartCookie);

        response.setContentType("text/html");
        response.getWriter().println("<h3>Item added! Current cart: " + updatedItems + "</h3>");
    }
}
```

Behaviour: Even if the user closes the search page and returns within 10 minutes, the browser automatically sends the 'cartItems' cookie back, so the cart is remembered. After 10 minutes the cookie expires and the browser stops sending it.

Q2. login.html + LoginServlet (HttpSession) + dashboard.jsp

login.html (see attached file) submits to LoginServlet via POST.

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

public class LoginServlet extends HttpServlet {

    private static final String VALID_USER = "zomatoUser";
    private static final String VALID_PASS = "pass123";

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (VALID_USER.equals(username) && VALID_PASS.equals(password)) {
            HttpSession session = request.getSession(); // creates a session if none exists
            session.setAttribute("username", username);

            response.sendRedirect("dashboard.jsp");
        } else {
            response.setContentType("text/html");
            response.getWriter().println("<h3>Invalid username or password</h3>");
        }
    }
}
```

dashboard.jsp (attached separately) reads session.getAttribute("username") and prints "Welcome, <username>!" using JSP scriptlet code.

Q3. URL Rewriting — Passing Username from LoginServlet to ProfileServlet

Instead of (or in addition to) storing the username in a session, it is appended directly to the redirect URL as a query parameter, and read back with `request.getParameter()`.

```
// Inside LoginServlet, after successful login:
String username = request.getParameter("username");

// URL rewriting: pass the username as a URL parameter
response.sendRedirect("ProfileServlet?user=" + username);
```

ProfileServlet.java:

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class ProfileServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("user");

        response.setContentType("text/html");
        response.getWriter().println("<h2>Profile Page</h2>");
        response.getWriter().println("<p>Username: " + username + "</p>");
    }
}
```

Note: URL rewriting works even if cookies are disabled in the browser, since the data travels as part of the URL itself rather than relying on cookies/session IDs.

Q4. LogoutServlet — Invalidating the Session

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

public class LogoutServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false); // don't create a new one
        if (session != null) {
            session.invalidate(); // destroys all session data
        }

        response.sendRedirect("login.html");
    }
}
```

The Logout button on dashboard.jsp (see the attached file) is a small form that POSTs to LogoutServlet, which invalidates the session and redirects back to the login page.

Q5. 'Remember Me' Login (Zomato-style)

The login logic is extended so that, when 'Remember Me' is checked, the session is set to stay alive for a long period instead of the default short timeout — simulating staying logged in until the user explicitly logs out or closes the browser.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");
    String rememberMe = request.getParameter("rememberMe"); // "yes" if checkbox ticked

    if (VALID_USER.equals(username) && VALID_PASS.equals(password)) {
        HttpSession session = request.getSession();
        session.setAttribute("username", username);

        if ("yes".equals(rememberMe)) {
            // Keep the session alive much longer (e.g. 7 days = 604800 seconds)
            session.setMaxInactiveInterval(7 * 24 * 60 * 60);
        } else {
            // Default/shorter timeout, e.g. 15 minutes of inactivity
            session.setMaxInactiveInterval(15 * 60);
        }

        response.sendRedirect("dashboard.jsp");
    } else {
        response.getWriter().println("<h3>Invalid username or password</h3>");
    }
}

/*
 * How session timeout affects this behaviour:
 * - setMaxInactiveInterval() only controls how long the session survives on the
 *   SERVER without any activity from that user. It does NOT keep the browser open
 *   or guarantee the cookie survives a browser close, unless a persistent cookie
 *   is also used for the session ID.
 * - If "Remember Me" is checked but the browser is fully closed and the session
 *   cookie (JSESSIONID) was a normal (non-persistent) cookie, the session ID is
 *   lost anyway, and the server-side session eventually expires on its own after
 *   the configured timeout even though no one is using it.
 * - If the timeout is too SHORT, the user gets logged out quickly even with
 *   "Remember Me" checked, defeating the purpose.
 * - If the timeout is too LONG (as set above for Remember Me), the session stays
 *   valid and the user stays logged in across visits, closely matching real apps
 *   like Zomato, until they click Logout (session.invalidate()) or the timeout
 *   is finally reached.
 */
}
```